CubeCanvas: Rubik Cube Mosaic Generator
Introduction to Image Processing

Contents

1 Introduction

CubeCanvas is an image processing application designed to transform ordinary digital images into Rubik Cube mosaics.

The application combines concepts from:

- Digital Image Processing
- Computer Vision
- Numerical Computing
- Graphical User Interfaces
- Object Oriented Programming
- PDF Generation

The user imports an image, selects a crop region, applies enhancement operations, converts the image into Rubik Cube colors, and finally exports printable assembly instructions.

1.1 Project Objectives

The primary objectives are:

- Import and display images.
- Perform interactive cropping.
- Apply image enhancement techniques.
- Convert images to Rubik Cube color palettes.
- Generate mosaic representations.
- Export assembly instructions as PDF.

1.2 Applications

Applications include:

- Rubik Cube Art
- Pixel Art Creation
- Educational Demonstrations
- Color Quantization Studies
- Image Processing Projects

2 Fundamentals of Digital Images

2.1 Digital Images

A digital image is a numerical representation of a visual scene.

An image consists of tiny picture elements called pixels.

$$Pixel = Picture\ Element$$

Each pixel stores brightness and color information.

2.2 Pixel Grid

Images are arranged in rectangular grids.

For example:

$$1920 \times 1080$$

contains

$$1920 \times 1080 = 2,073,600$$

pixels.

Mathematically:

$$I(x, y)$$

represents the pixel located at coordinates

$$(x, y)$$

inside an image.

2.3 Grayscale Images

A grayscale image stores one intensity value per pixel.

$$0 \leq I(x, y) \leq 255$$

where:

- 0 = Black
- 255 = White
- Intermediate values = Shades of Gray

Example:

$$\begin{bmatrix} 0 & 50 & 100 \\ 150 & 200 & 255 \end{bmatrix}$$

2.4 RGB Color Space

Color images use three channels:

- Red
- Green
- Blue

Each channel ranges from:

$$0 \to 255$$

Therefore:

$$(R, G, B)$$

represents a pixel.
Examples:

$$(255, 0, 0)$$

Pure Red

$$(0, 255, 0)$$

Pure Green

$$(0, 0, 255)$$

Pure Blue

$$(255, 255, 255)$$

White

$$(0, 0, 0)$$

Black

2.5 Why RGB Works

Human eyes contain receptors sensitive to red, green, and blue wavelengths.
Combining these colors creates almost every visible color.
Examples:

$$(255, 255, 0)$$

Yellow

$$(255, 0, 255)$$

Magenta

$$(0, 255, 255)$$

Cyan

2.6 NumPy Image Representation

In CubeCanvas:

```
arr = np.array(image)
```

An image of size

$$300 \times 400$$

becomes

```
(300,400,3)
```

meaning:

- Height = 300
- Width = 400
- RGB Channels = 3

Mathematically:

$$Image \in \mathbb{R}^{H \times W \times 3}$$

2.7 Pixel Access

Individual pixels:

```
arr[y,x]
```

Example:

```
arr[10,20]
```

returns:

```
[255,128,50]
```

2.8 Image Loading

Images are loaded using Pillow.

```
image = Image.open(path)
```

Supported formats:

- PNG
- JPG
- JPEG
- BMP
- TIFF

3 Image Manipulation and Resizing

3.1 Image Cropping

Cropping extracts a region from an image.

```
cropped = image.crop(
(x1,y1,x2,y2))
```

Mathematically:

$$I_c(x, y) = I(x + x_0, y + y_0)$$

where

$$(x_0, y_0)$$

is the crop origin.

3.2 Region of Interest (ROI)

The selected crop area is called the Region of Interest.

Benefits:

- Reduces computation
- Removes unwanted regions
- Focuses processing

3.3 Aspect Ratio

Aspect Ratio:

$$AspectRatio = \frac{Width}{Height}$$

Example:

$$1920 \times 1080$$

$$\frac{1920}{1080} = 1.777$$

or

$$16 : 9$$

Maintaining aspect ratio prevents distortion.

3.4 Image Resizing

Image resizing changes dimensions.

$$I(x, y) \to I'(x', y')$$

Example:

$$1000 \times 1000$$

becomes

$$300 \times 300$$

3.5 Upsampling

Increasing image size.
Example:

$$100 \times 100 \to 1000 \times 1000$$

New pixels must be estimated.

3.6 Downsampling

Reducing image size.
Example:

$$1000 \times 1000 \to 100 \times 100$$

Information is discarded.
CubeCanvas primarily uses downsampling before mosaic generation.

3.7 Interpolation Techniques

Interpolation estimates unknown pixel values.

3.7.1 Nearest Neighbor

Used in:

`Image.Resampling.NEAREST`

Formula:

$$I'(x, y) = I(round(x), round(y))$$

Advantages:

- Fast
- Preserves sharp edges
- Good for pixel art

3.7.2 Bilinear Interpolation

Uses four neighboring pixels.

$$f(x, y) = \sum w_i p_i$$

Produces smoother results.

3.7.3 Bicubic Interpolation

Uses sixteen neighboring pixels.
Advantages:

- Better quality
- Smooth transitions
- Reduced artifacts

3.7.4 Lanczos Interpolation

Used in CubeCanvas.

`Image.Resampling.LANCZOS`

Based on the sinc function:

$$sinc(x) = \frac{\sin(\pi x)}{\pi x}$$

Lanczos Kernel:

$$L(x) = sinc(x) \cdot sinc\left(\frac{x}{a}\right)$$

Advantages:

- Excellent quality
- Sharp details
- Minimal aliasing

4 Color Quantization and Dithering

4.1 Introduction

A standard RGB image can contain millions of colors.
Since each RGB channel can take:

$$256$$

possible values, the total number of colors becomes:

$$256^3 = 16,777,216$$

colors.

However, a Rubik Cube supports only a limited set of colors.

Therefore, the image must be converted from millions of colors into a small predefined palette.

This process is called Color Quantization.

4.2 Color Quantization

Color Quantization is the process of reducing the number of colors in an image while preserving visual appearance as much as possible.

Mathematically:

$$Q(I)$$

represents the quantized version of image

$$I$$

where:

$$I \rightarrow Q(I)$$

The objective is:

$$16.7M\ Colors \rightarrow 6\ Rubik\ Colors$$

4.3 Rubik Cube Color Palette

CubeCanvas uses six primary Rubik Cube colors.

```
palette = [
(255,255,255),
(255,255,0),
(255,0,0),
(255,136,0),
(0,0,255),
(0,170,0)
]
```

These represent:

- White
- Yellow
- Red
- Orange
- Blue
- Green

Mathematically:

$$P = \{p_1, p_2, p_3, p_4, p_5, p_6\}$$

where:

$$p_i = (R_i, G_i, B_i)$$

4.4 RGB Color Space Geometry

RGB colors can be interpreted as points in a three-dimensional coordinate system.
Axes:

- X-axis = Red
- Y-axis = Green
- Z-axis = Blue

Each color becomes a point:

$$(R, G, B)$$

Examples:

$$(255, 0, 0)$$

Red

$$(0, 255, 0)$$

Green

$$(0, 0, 255)$$

Blue

$$(255, 255, 255)$$

White
This transforms color matching into a geometric distance problem.

4.5 Euclidean Distance

Suppose:

$$c = (R, G, B)$$

is a pixel color.
And

$$p = (r, g, b)$$

is a palette color.

Distance between them:

$$d = \sqrt{(R-r)^2 + (G-g)^2 + (B-b)^2}$$

The palette color with minimum distance is selected.

4.6 Squared Distance Optimization

Computing square roots repeatedly is expensive.

Since:

$$\sqrt{x}$$

is monotonic,

$$x_1 < x_2 \Rightarrow \sqrt{x_1} < \sqrt{x_2}$$

therefore:

$$(R-r)^2 + (G-g)^2 + (B-b)^2$$

can be compared directly.

This optimization is used in the project.

4.7 Nearest Color Selection

Implementation:

```
def nearest_color(color,palette):

    distances =
    np.sum(
    (palette-color)**2,
    axis=1)

    return palette[
    np.argmin(distances)]
```

Algorithm:

1. Compute distance to every palette color.
2. Store distances.
3. Find minimum distance.
4. Return nearest palette color.

4.8 Worked Example

Consider:

$$c = (250, 20, 20)$$

Distance from red:

$$(255, 0, 0)$$

$$d_r^2 = (250 - 255)^2 + (20 - 0)^2 + (20 - 0)^2$$

$$= 25 + 400 + 400 = 825$$

Distance from white:

$$d_w^2 = (250 - 255)^2 + (20 - 255)^2 + (20 - 255)^2$$

$$= 25 + 55225 + 55225 = 110475$$

Since:

$$825 < 110475$$

the pixel becomes red.

4.9 Problem of Simple Quantization

Suppose a grayscale gradient:

$$0 \to 255$$

After quantization:

$$0 \to 0 \to 255 \to 255$$

Visible color bands appear.
This effect is called:

Color Banding

Banding reduces image quality.

4.10 Dithering

Dithering is used to reduce banding artifacts.

Instead of independently replacing each pixel, quantization errors are distributed to neighboring pixels.

Advantages:

- Smoother gradients
- Better visual quality
- Reduced banding

4.11 Floyd-Steinberg Dithering

CubeCanvas uses Floyd-Steinberg Error Diffusion.
Algorithm:

1. Quantize current pixel.
2. Compute error.
3. Distribute error.
4. Continue scanning image.

4.12 Quantization Error

Original pixel:

$$Old$$

Quantized pixel:

$$New$$

Error:

$$Error = Old - New$$

Example:

$$Old = (120, 120, 120)$$

$$New = (100, 100, 100)$$

$$Error = (20, 20, 20)$$

This information should not be discarded.

4.13 Error Diffusion Matrix

Floyd-Steinberg distributes error using:

$$\begin{array}{ccc} & * & \frac{7}{16} \\ \frac{3}{16} & \frac{5}{16} & \frac{1}{16} \end{array}$$

where

$$*$$

represents the current pixel.
Notice:

$$\frac{7}{16} + \frac{3}{16} + \frac{5}{16} + \frac{1}{16} = 1$$

Thus the complete error is preserved.

4.14 Error Propagation Equations

For error

$$e$$

Current pixel:

$$(x, y)$$

Updates:

$$I(x+1, y) = I(x+1, y) + \frac{7}{16}e$$

$$I(x-1, y+1) = I(x-1, y+1) + \frac{3}{16}e$$

$$I(x, y+1) = I(x, y+1) + \frac{5}{16}e$$

$$I(x+1, y+1) = I(x+1, y+1) + \frac{1}{16}e$$

4.15 Python Implementation

```
if x+1 < w:
    arr[y,x+1] += err*7/16

if y+1 < h and x>0:
    arr[y+1,x-1] += err*3/16

if y+1 < h:
    arr[y+1,x] += err*5/16

if y+1 < h and x+1 < w:
    arr[y+1,x+1] += err*1/16
```

4.16 Why Dithering Works

The human eye performs spatial averaging.

Instead of seeing individual pixels, nearby colors are visually blended.

This creates the illusion of intermediate colors.

As a result:

- Gradients appear smoother.
- Images look more natural.
- Fewer quantization artifacts appear.

4.17 NumPy Processing Pipeline

Image:

$$PIL\ Image$$

$$\downarrow$$

$$NumPy\ Array$$

using:

```
arr = np.array(img)
```

Each pixel:

```
arr[y,x]
```

contains:

```
[R,G,B]
```

The dithering algorithm iterates through every pixel.

4.18 Pixel Clipping

Error propagation may generate invalid values.
Example:

$$-10$$

or

$$300$$

These are not valid RGB values.
Therefore:

```
np.clip(arr,0,255)
```

is used.
Mathematically:

$$f(x) = \begin{cases} 0, & x < 0 \\ x, & 0 \leq x \leq 255 \\ 255, & x > 255 \end{cases}$$

4.19 Mosaic Resolution

Each Rubik Cube contains:

$$3 \times 3$$

stickers.
Therefore:

```
cols *= 3
rows *= 3
```

Example:

$$15 \times 15$$

cubes become:

$$45 \times 45$$

pixels.
General formula:

$$Pixels = 3 \times Cubes$$

4.20 Mosaic Generation Pipeline

Complete workflow:
Image
↓
Crop
↓
Resize
↓
Convert RGB
↓
NumPy Array
↓
Nearest Color Search
↓
Floyd-Steinberg Dithering
↓
Rubik Palette Image
↓
Mosaic Output

4.21 Complexity Analysis

Let:

$$N = Number\ of\ Pixels$$

$$K = Palette\ Size$$

Nearest color search:

$$O(K)$$

per pixel.
Total complexity:

$$O(NK)$$

Since:

$$K = 6$$

the algorithm is highly efficient.

5 Image Enhancement Techniques

5.1 Introduction

Image Enhancement refers to techniques used to improve the visual quality of an image.

Unlike image compression or storage, enhancement focuses on improving the appearance of the image for human observation.

Typical objectives include:

- Improving visibility
- Increasing contrast
- Enhancing colors
- Highlighting details
- Removing noise
- Preparing images for further processing

In CubeCanvas, enhancement operations are performed before color quantization.

Processing pipeline:

Image
↓
Brightness
↓
Contrast
↓
Saturation
↓
Sharpness
↓
Blur
↓
Flip Operations
↓
Color Quantization
↓
Mosaic Generation

5.2 Image Enhancement Pipeline

Enhancement operations are implemented using Pillow.

Simplified code:

```
img = ImageEnhance.Brightness(
img).enhance(brightness)

img = ImageEnhance.Contrast(
img).enhance(contrast)

img = ImageEnhance.Color(
img).enhance(saturation)

img = ImageEnhance.Sharpness(
img).enhance(sharpness)

img = img.filter(
ImageFilter.GaussianBlur(radius))
```

Each operation modifies pixel values mathematically.

5.3 Brightness Adjustment

Brightness determines how light or dark an image appears.

Every pixel intensity is multiplied by a constant value.

Mathematical model:

$$I'(x, y) = kI(x, y)$$

where:

- $I(x, y)$ = original pixel intensity
- $I'(x, y)$ = transformed intensity
- k = brightness factor

Interpretation:

- $k > 1$ increases brightness
- $k < 1$ decreases brightness
- $k = 1$ leaves image unchanged

Example:

$$I = 100$$

For:

$$k = 2$$

$$I' = 200$$

For:

$$k = 0.5$$

$$I' = 50$$

Brightness affects all RGB channels equally.
Example:

$$(100, 150, 200)$$

with

$$k = 1.5$$

becomes

$$(150, 225, 300)$$

After clipping:

$$(150, 225, 255)$$

5.4 Brightness Histogram Effect

A histogram represents the distribution of intensity values.
Brightness shifts the histogram.
Original range:

$$50 \rightarrow 200$$

After increasing brightness:

$$100 \rightarrow 250$$

Thus the histogram moves toward larger values.

5.5 Contrast Enhancement

Contrast measures the difference between bright and dark regions.
High contrast images have strong intensity differences.
Low contrast images appear flat and dull.
Mathematical model:

$$I' = \alpha(I - 128) + 128$$

where:

- I = original intensity

- I' = transformed intensity
- α = contrast factor
- 128 = midpoint intensity

Interpretation:

- $\alpha > 1$ increases contrast
- $\alpha < 1$ decreases contrast
- $\alpha = 1$ leaves image unchanged

Example:

$$I = 200$$

$$\alpha = 1.5$$

Then:

$$I' = 1.5(200 - 128) + 128$$

$$= 236$$

Bright pixels become brighter.
Now:

$$I = 50$$

$$I' = 1.5(50 - 128) + 128$$

$$= 11$$

Dark pixels become darker.

5.6 Histogram Stretching

Contrast enhancement increases histogram spread.
Before:

$$100 \rightarrow 150$$

After:

$$50 \rightarrow 200$$

Benefits:

- Better visibility
- Enhanced details
- Improved edge perception

5.7 Color Saturation

Saturation controls color intensity.

It determines how vivid or dull colors appear.

High saturation:

- Rich colors
- Vibrant appearance

Low saturation:

- Muted colors
- Gray appearance

5.8 Grayscale Conversion

Before understanding saturation, we must understand grayscale intensity.

A grayscale approximation is:

$$Gray = 0.299R + 0.587G + 0.114B$$

These coefficients model human visual perception.

The eye is most sensitive to green.

5.9 Saturation Mathematics

A simplified saturation model:

$$I' = Gray + s(I - Gray)$$

where:

- $Gray$ = grayscale intensity
- s = saturation factor

Interpretation:

- $s = 0 \rightarrow$ grayscale image
- $s = 1 \rightarrow$ original image
- $s > 1 \rightarrow$ enhanced colors

Example:

$$RGB = (200, 50, 50)$$

Approximate grayscale:

$$Gray = 95$$

If:

$$s = 0$$

Result:

$$(95, 95, 95)$$

If:

$$s = 2$$

Colors become significantly more vivid.

5.10 Sharpness Enhancement

Sharpness controls edge visibility.

Edges correspond to rapid intensity changes.

Example:

$$10, 10, 10, 250, 250, 250$$

The sudden transition represents an edge.

Sharpening increases these transitions.

5.11 Edge Information

Edges contain important information about object boundaries.

Applications:

- Object detection
- Shape recognition
- Feature extraction
- Visual perception

5.12 Unsharp Masking

Most sharpening algorithms use:

$$Sharpened = Original + k(Original - Blurred)$$

This technique is called Unsharp Masking.

Steps:

1. Blur image
2. Subtract blurred image
3. Extract edge information
4. Add edges back

5.13 Why Unsharp Masking Works

Blur removes high-frequency information.

Therefore:

$$Edge = Original - Blurred$$

Extracts edges.

Adding edges back:

$$Enhanced = Original + Edge$$

produces a sharper image.

5.14 Convolution

Many image filters are based on convolution.

Convolution combines neighboring pixel information.

General equation:

$$G(x, y) = \sum f(i, j)h(x - i, y - j)$$

where:

- f = image
- h = kernel
- G = output image

5.15 Convolution Kernel

A kernel is a small matrix used for filtering.

Example sharpening kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

The kernel slides across the image and computes new pixel values.

5.16 Gaussian Blur

Blur smooths the image.

Small details and noise are reduced.

CubeCanvas uses:

```
ImageFilter.GaussianBlur(radius)
```

5.17 Gaussian Distribution

Gaussian Blur is based on:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

This function creates a bell-shaped weighting curve.
Pixels closer to the center receive larger weights.

5.18 Gaussian Kernel

A common Gaussian kernel:

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Center pixels contribute most strongly.

5.19 Effects of Gaussian Blur

Advantages:

- Removes noise
- Smooths textures
- Reduces small details
- Creates softer appearance

Disadvantages:

- Loss of sharpness
- Reduced edge visibility

5.20 Frequency Domain Interpretation

Images contain frequency information.
Low frequencies:

- Smooth regions
- Large structures

High frequencies:

- Edges
- Fine details
- Noise

Gaussian Blur behaves as a:

Low Pass Filter

It removes high-frequency information.

5.21 Image Flipping

CubeCanvas supports image flipping operations.
Types:

- Horizontal Flip
- Vertical Flip

5.22 Horizontal Flip

Implemented using:

`ImageOps.mirror(img)`

Transformation:

$$x' = W - 1 - x$$

where:

$$W$$

is image width.
Example:

$$A \quad B \quad C$$

becomes

$$C \quad B \quad A$$

5.23 Vertical Flip

Implemented using:

`ImageOps.flip(img)`

Transformation:

$$y' = H - 1 - y$$

where:

$$H$$

is image height.
Example:

$$A$$

$$B$$

$$C$$

becomes

$$C$$

$$B$$

$$A$$

5.24 Order of Operations

CubeCanvas applies operations in the following order:

$$Brightness$$

$$\downarrow$$

$$Contrast$$

$$\downarrow$$

$$Saturation$$

$$\downarrow$$

$$Sharpness$$

$$\downarrow$$

$$Blur$$

$$\downarrow$$

$$Flip$$

Each operation modifies the output of the previous stage.

5.25 Computational Complexity

Let:

$$N = Number\ of\ Pixels$$

Brightness:

$$O(N)$$

Contrast:

$$O(N)$$

Saturation:

$$O(N)$$

Sharpness:

$$O(N)$$

Gaussian Blur:

$$O(Nk^2)$$

where:

$$k$$

represents kernel size.

6 PDF Export and Mosaic Generation

6.1 Introduction

After image processing is complete, the mosaic must be converted into printable instructions.

This functionality is implemented in:

`export_generator.py`

6.2 Need for Export

Large mosaics are difficult to assemble directly.

Therefore the image is divided into smaller sections.

Benefits:

- Easier assembly
- Reduced confusion
- Better organization

6.3 Mosaic Partitioning

The mosaic is divided into chunks.

Chunk size:

$$9 \times 9$$

pixels.

Reason:

Each chunk corresponds to:

$$3 \times 3$$

Rubik Cubes.

6.4 Chunk Extraction

Individual chunks are extracted from the image.

Coordinates:

$$(x_1, y_1)$$

to

$$(x_2, y_2)$$

Each chunk becomes one instruction page.

6.5 Page Generation

For each chunk:

1. Extract chunk
2. Generate preview
3. Draw cube boundaries
4. Draw sticker colors
5. Add labels

A complete instruction page is created.

6.6 Cube Visualization

Each Rubik Cube contains:

$$3 \times 3$$

stickers.

Each sticker is drawn using:

```
draw.rectangle(...)
```

Sticker colors correspond to quantized pixel values.

6.7 Overview Page

An overview page is generated before individual instruction pages.

Purpose:

- Display complete mosaic
- Show section boundaries
- Provide navigation map

6.8 Coordinate Labeling

Sections are labeled using coordinates.

Columns:

$$A, B, C, D, \ldots$$

Rows:

$$1, 2, 3, 4, \ldots$$

Examples:

$$A1$$

$$B3$$

$$D5$$

This allows users to locate sections easily.

6.9 PDF Generation

Generated pages are stored in memory.

Finally:

$$Pages \rightarrow PDF$$

The user receives a printable assembly guide.

6.10 Memory Management

The application uses multiple image representations.

- PIL Images
- NumPy Arrays
- PhotoImage Objects

Each representation is optimized for a specific task.

6.11 Complete Workflow

The complete application workflow is:

Import Image
↓
Crop Image
↓
Apply Enhancements
↓

Resize
↓
Quantization
↓
Dithering
↓
Mosaic Generation
↓
Preview
↓
Chunk Division
↓
PDF Generation
↓
Export

6.12 Advantages of the System

The final system demonstrates:

- Digital Image Processing
- Numerical Computing
- GUI Design
- Event Driven Programming
- Object Oriented Programming
- PDF Generation
- Rubik Cube Mosaic Construction